תכנות מונחה עצמים – תרגיל 1

Tic-Tac-Toe Tournament

תאריך ההגשה: 16.11.25 בשעה 23:55

שימו לב! תרגיל זה יש להגיש רק ביחידים (לא בזוגות).

שימו לב: אין צורך לשאול שאלות נוספות לגבי הנחת תקינות הקלט: מה שלא מופיע בתרגיל - אין צורך לטפל בו.

0. הקדמה

בתרגיל זה נממש טורניר של משחקי איקס עיגול.

בגרסה שלנו הלוח יהיה בגודל $n \times n$, וסיבוב יחיד של משחק נגמר כאשר יש שחקן מנצח או כאשר לא נותרו משבצות ריקות בלוח. שחקן מוגדר כמנצח אם הוא השיג על הלוח רצף של k משבצות מסומנות בסימן שלו (X או O). הרצף יכול להיות מאונך, מאוזן או אלכסוני (הן יורד מימין לשמאל והן יורד משמאל לימין). כאשר n ו-k הם משתנים עם ערכים שיבחר המשתמש, ועם הערכים הדיפולטיביים 4 ו-3 בהתאמה. נשים לב, שבשביל שיהיה משמעות למשחק, נדרוש $2 \leq k \leq n$.

בתרגיל יהיו שני סוגי שחקנים, שחקן אנושי שישחק את תורו דרך ה-`System.in`; ושחקן אוטומטי, שיפעל באופן אוטומטי לפי אסטרטגיה מוגדרת.

1. מהלך המשחק

1.1. בתחילת המשחק יבחר המשתמש שני סוגי שחקנים, השחקנים האלה יכולים להיות מאותו סוג - שניהם אוטומטיים/אנושיים, או מסוגים שונים - אחד אוטומטי ואחד אנושי.

1.2. השחקנים שנבחרו יערכו טורניר המורכב ממספר סיבובים שיגדיר המשתמש.
בכל סיבוב יחליפו השחקנים תפקיד כך שישחקו לסירוגין בסימנים איקס ועיגול, ולוח התוצאות יציג כמה סיבובים ניצח שחקן 1, כמה סיבובים ניצח שחקן 2 וכמה סיבובים הסתיימו בתיקו.

1.3. כמו כן, יבחר המשתמש משתנה שלישי שמגדיר האם הלוח ירונדר על המסך או לא.
האפשרות לא להציג את הלוח שימושית במיוחד כשרוצים להריץ טורניר של מספר סיבובים בין שחקנים אוטומטיים.

2. שורת ההרצה

2.1. משתמש שרוצה להריץ את המשחק, יקליד את הפקודה הבאה:

```
java Tournament [round count] [size] [win_streak]
[render target: console/void]
[first player: human/whatever/naive/smart]
[second player: human/whatever/naive/smart]
```

לדוגמא, הפקודה הבאה תריץ טורניר של 10,000 סיבובים על לוח בגודל 4×4, ורצף ניצחון של 3, בין השחקנים `Whatever` ו `Naive` (ר׳ פירוט למטה לגבי סוגי השחקנים), ולא תדפיס את הלוח על המסך:

```
java Tournament 10000 4 3 void whatever naive
```

כדי לשחק שלושה משחקים בין שחקן אוטומטי למשתמש, הפקודה היא:

```
java Tournament 3 4 3 console whatever human
```

2.2. במקרה שבו תבחרו בערך `void` עבור רינדור הלוח ותגדירו גם שחקן אנושי, תקבלו בדיוק את מה שביקשתם: הלוח לא יודפס על המסך אבל כשיגיע תורו של המשתמש האנושי הוא עדיין יתבקש לבחור משבצת על לוח המשחק.

2.3. <u>הערה חשובה מאוד:</u> בניגוד לשפות אחרות, כמו למשל C ופייתון, בשפת java הארגומנטים משורת הפקודה ("argv") **לא מתחילים מאינדקס 1**.
כלומר, אם תרשמו את הפקודה הבאה בעת ההרצה:

Java Tournament I1 I2 I3 S1 S2 S3

אז הארגומנט במקום 0 הוא I1 (ולא "java"), במקום 1 הוא I2 (ולא "Tournament"), במקום 2 הוא I3 (ולא "I1") וכן הלאה.
ייתכן שב-IntelliJ לא יוצג כך ולכן הקפידו להריץ גם משורת הפקודה.

2.4. ניתן להניח **שכל הקלט משורת ההרצה תקין**. למשל, שאת התוכנית מריצים עם 6 ארגומנטים, שכולם חוקיים, בסדר נכון, שגודל הלוח הוא ספרה בין 2-9, גם רצף הניצחון (וגם שהוא קטן מגודל הלוח) - ועוד.

API.3

בגדול:נציג את ה-*API* של המשחק המורכב משני ממשקים, שלוש מחלקות משחק, שתי מחלקות רינדור, ארבע מחלקות שחקן, שתי מחלקות מפעל ומ-Enum אחד:

- ממשקים:
  - `Renderer`
  - `Player`
- מחלקות:
  - `Board`
  - `VoidRenderer`
  - `ConsoleRenderer` – המימוש שלה נמצא ב-Moodle.
  - `KeyboardInput` – המימוש שלה נמצא ב-Moodle.
  - `HumanPlayer`
  - `NaivePlayer`
  - `WhateverPlayer`
  - `SmartPlayer`
  - `Game`
  - `Tournament`
  - `PlayerFactory`
  - `RendererFactory`
- ★ `enum Mark`

הערות:

- שימו לב שהמחלקה `ConsoleRenderer`, כמו גם המחלקה KeyboardInput, מסופקות לכם במודל, ואין צורך לממש אותן בעצמכם.
- כל המתודות שנציג כחלק מה *API* יהיו בעלות מגדיר נראות פומבי (`public`).
- הגדירו פונקציות עזר פרטיות ושדות פרטיים לפי הצורך.

טיפ: התחילו קודם כל ביצירת הממשקים.

להלן תיאור מפורט של המחלקות הממשקים וה-enum בתרגיל:

3.1. `Mark`:

`Enum` בשם `Mark` ייצג את הסימונים על הלוח, והוא יוגדר כך:

```
enum Mark{BLANK, X, O}
```

והוא ישב בקובץ `Mark.java`.

בנוסף, עליכם ליצור מתודה בשם **`toString`** ב-`Enum`. מטרתה של המתודה היא המרת הסימון הנבחר ב-`Enum` ל-`String`. תוכלו להשתמש בה על מנת להדפיס את הסימון בקלות.

במקרה שה-`Enum` הנבחר הוא `BLANK`, ניתן להחזיר `null`.

חתימת המתודה היא:

```
String toString()
```

3.2. המחלקה `Board`:

אחראית על מצב הלוח: גודל הלוח, סימון משבצות ושמירת כל מה שסומן על הלוח. שימו לב כי מספור המשבצות על הלוח מתחיל ב-0.

רשימת המתודות של המחלקה:

מתודה	משמעות
`Board()`	בנאי דיפולטיבי, מגדיר לוח ריק חדש בגודל הדיפולטיבי.
`Board(int size)`	בנאי נוסף, מגדיר לוח ריק חדש בגודל $size * size$.
`int getSize()`	מחזירה את גודל הלוח, כלומר אורך שורה או אורך עמודה (הלוח תמיד בצורה של $n * n$.)
`boolean putMark(Mark mark, int row, int col)`	תנסה לסמן את המשבצת (row, col) ב $mark$ הנבחר. מחזירה $True$ אם סימנה את המשבצת בהצלחה (לפי כללי המשחק הסטנדרטיים).
`Mark getMark(int row, int col)`	מחזירה את הסימון שיש במשבצת (row, col). במקרה של קבלת קואורדינטות לא חוקיות תחזיר $Mark.BLANK$

3.3. הממשק `Renderer` ומחלקות הרינדור:

ממשק שמייצג צורה להצגת מהלך של משחקי איקס עיגול על המסך.

מתודה	משמעות
`void renderBoard(Board board)`	קבל לוח והצג אותו לשיטתך.

3.3.1. המחלקה `ConsoleRenderer`:

את המחלקה הזו אתם אינכם ממשים או משנים. המימוש של מחלקה זו נמצא בMoodle הקורס, אין לשנות או לערוך אותה - גם לא מגישים את הקובץ.

המחלקה מייצגת צורה של הצגה ויזואלית של מהלך משחק על הלוח על ידי הדפסות לטרמינל. ה API של המחלקה מכיל שתי מתודות:

- בנאי דיפולטיבי.
- מתודה עם החתימה: $void\ renderBoard(Board\ board)$, שמקבלת לוח ומציגה אותו על המסך.

3.3.2. המחלקה `VoidRenderer`:

המחלקה מממשת (implements) את הממשק `Renderer`, אבל מהווה מימוש ריק של `Renderer` שלמעשה לא מציג כלום על המסך, אך זה עדיין נחשב צורה של רינדור. בעזרת המחלקה הזו, ניתן להריץ משחק בין שחקנים שתממשו מבלי לראות הדפסות של הלוח על המסך.

3.4. הממשק `Player`:

ממשק שמייצג לוגיקה או אסטרטגיה מסוימות לביצוע תור במשחק בהינתן לוח מסוים.

משמעות	מתודה
מבצעת את האסטרטגיה של השחקן.	`void playTurn(Board board, Mark mark)`

3.4.1. **שימו לב:** לא ניתן להוסיף לממשק פונקציות נוספות! הוספה של פונקציה נוספת תגרום לכישלון הקומפילציה של הטסטים ולקבלת 0 בתרגיל.

3.4.2. ארבעת המחלקות `NaivePlayer`, `WhateverPlayer`, `SmartPlayer` ו-`HumanPlayer` יממשו את `Player`.

3.4.3. בנוסף לשחקן האנושי, תממשו שלושה שחקנים אוטומטיים. השחקנים האוטומטיים אמורים לתת תוצאה מסויימת, ואנו מצפים מכם לחשוב איך להשיג אותה.

- המהלכים של `WhateverPlayer` אקראיים.
- `SmartPlayer` תנצח את `WhateverPlayer` ברוב המוחלט של המשחקים.
- `SmartPlayer` תנצח את `NaivePlayer` ברוב המוחלט של המשחקים.

במילים אחרות, אנו דורשים שתממשו לוגיקה לשחקנים האוטומטיים כך שברוב המוחלט של המשחקים יתקיים `WhateverPlayer, NaivePlayer < SmartPlayer`. <u>הסבר מדויק על כמות הנצחונות רשום בתיאור המחלקות עצמן בסעיפים הבאים.</u>

שימו לב שהמחלקות המממשות את הממשק `Player` לא תלויות במשחק מסוים; מופע אחד של שחקן יכול לקחת חלק במשחק אחד או בכמה, ועם סימונים שונים. כתוצאה מכך, הבנאי של כל אחד מהמחלקות שממשות את הממשק `Player` לא דורשות שום קלט, היות ולא נדרש אף מידע לגבי המשחק או הלוח כדי לאתחל את השחקן.

3.5. המחלקה `HumanPlayer`:

המחלקה מייצגת שחקן אנושי. האחריות היחידה של מחלקה זו היא בקשת קלט מהמשתמש.

רשימת המתודות של המחלקה:

משמעות	מתודה
בנאי, מגדיר שחקן חדש	**`HumanPlayer()`**
מבקשת קואורדינטות מהמשתמש וממקמת את הסימן במידה והקואורדינטות "טובות" - כלומר תקינות ולא תפוסות; במקרה והקלט לא תקין, היא תדפיס על כך הודעה ותצפה לקלט חדש (אופן ההדפסות יתואר בהמשך).	**`void playTurn(Board board, Mark mark)`**

<u>אופן ההדפסות והנחיית השחקן:</u>

3.5.1. כדי לבקש קלט, המחלקה תדפיס בהתאם ל-mark של השחקן:

Player X, type coordinates:

או:

Player O, type coordinates:

ניתן (וניתן גם שלא) לרשום רווח לאחר הנקודותיים.

3.5.2. אופן קבלת הקלט: מצופה מהשחקן האנושי להכניס מספר דו ספרתי, שספרת העשרות שלו מהווה מספר השורה, וספרת היחידות שלו מהווה מספר העמודה.
הספרות נמצאות בטווח $[0, Board.Size - 1]$ (כאשר "0" מהווה השורה/העמודה הראשונה, ו-$Board.Size - 1$ השורה/העמודה האחרונה).

> דוגמא לקלט חוקי: "31"≡ שורה רביעית, עמודה שניה. (כלומר בוחרים את 3 עבור השורה, ואת 1 עבור העמודה)

3.5.3. לשם קבלת קלט, **<u>חובה</u>** להשתמש אך ורק במחלקה `KeyboardInput` שניתנה לכם כחלק מקבצי ההגשה (היא מופיעה ב-moodle).
לקבלת קלט, יש לקרוא לפונקציה `KeyboardInput.readInt()` המחזירה ערך `int` שהקליד המשתמש.
תוכלו תמיד להניח שהקלט עצמו הוא מספר תקין (שאולי מהווה קורדינטות לא חוקיות).
אין להגיש את הקובץ `KeyboardInput`, **ואין** לשנות אותו.

3.5.4. אופן התמודדות עם קלט לא תקין:
ניתן להניח שהקלט מתקבל בצורה תקינה (מספר טבעי), אך לא ניתן להניח שהערכים נמצאים בטווח המצוין לעיל. למשל - אם גודל הלוח הנבחר הוא 6, **לא תקין** ששחקן יבחר במשבצת בשורה **7** ועמודה **5** על גבי הלוח.
במקרה כזה, נא להדפיס שהקלט לא חוקי עם ההודעה הבאה:

Invalid mark position. Please choose a valid position:

3.5.5. אם שחקן בוחר משבצת שיש עליו כבר סימון של שחקן כלשהו, כלומר המשבצת אינה blank, אז יש להדפיס:

Mark position is already occupied. Please choose a valid position:

3.5.6. בשתי המקרים, **אין להדפיס מחדש Player <mark>, type coordinates לאחר מכן.**
וגם כאן, ניתן (וניתן שלא) להוסיף רווח לאחר הנקודתיים.

3.5.7. כל ההודעות מודפסות כמובן ל-stdout (כלומר, השתמשו ב-System.out).

3.6. שחקנים אוטומטיים

המתודות הפומביות של השחקנים האוטומטיים זהות למתודות הפומביות של ה-`HumanPlayer` ולכן אנו לא מתארים אותן שוב.

3.6.1. המחלקה `WhateverPlayer`:

אסטרטגיית שחקן זה היא בחירה של משבצת אקראית ריקה על הלוח (אם אתם לא זוכרים איך בוחרים מספר אקראי, חזרו לשיעור 1.2).

3.6.2. המחלקה `NaivePlayer`:

שחקן זה מממש את האסטרטגיה הבאה:

- נעבור שורה שורה, לפי הסדר.
- בתוך כל שורה, נעבור על כל עמודה.
- אם המקום פנוי - נניח את הסימן שלנו.

כלומר, מניחים את הסימן במקום הראשון שפנוי על הלוח (כאשר בודקים את הלוח שורה אחרי שורה ובתוך כל שורה עמודה אחרי עמודה).

3.6.3. המחלקה `SmartPlayer`:

אסטרטגיה שמנצחת את שתי השחקנים WhateverPlayer ו-NaivePlayer ב"רוב הפעמים" (כפי שיתואר מיד).

גם כאן, עליכם לוודא את הצלחתו באופן הבא, בטורניר בין $10,000$ משחקים, **על לוח ורצף ניצחון בגדלים הדיפולטיביים:**

- בין השחקן לבין `WhateverPlayer`. **על המימוש שלכם ל-`SmartPlayer` לנצח לפחות ב-80% מהמשחקים.**
- בין השחקן לבין `NaivePlayer`. **על המימוש שלכם ל-`SmartPlayer` לנצח לפחות ב-80% מהמשחקים.**

3.7. המחלקה `Game`:

מופע של המחלקה מייצג משחק יחיד. עליו לדעת מתי המשחק נגמר, מי היה המנצח והאם הוא הסתיים בתיקו.

משמעות	מתודה
בנאי, מגדיר משחק חדש, עם ערכים דיפולטיביים.	**`Game(Player playerX,Player playerO, Renderer renderer)`**
בנאי נוסף, מגדיר לוח בגודל $size$, ורצף ניצחון באורך $winStreak$.	**`Game(Player playerX,Player playerO, int size, int winStreak,Renderer renderer)`**
מחזירה את אורך רצף הניצחון	**`int getWinStreak()`**
מחזירה את גודל הלוח	**`int getBoardSize()`**
מריצה מהלך של משחק בודד - מתחילתו ועד סופו, ומחזירה את הסימן של המנצח.	**`Mark run()`**

הנחיות:

3.7.1. הפרמטר $winStreak$ של הבנאי מהווה את הרצף שצריך (באלכסונים, בקו מאונך או מאוזן) כדי להשיג ניצחון.

3.7.2. התור הראשון של המשחק תמיד יהיה שייך לשחקן שסימנו X.

3.7.3. המשחק מסתיים כאשר לאחד מהשחקנים יש רצף ניצחון או כאשר לא נותרו משבצות ריקות בלוח. במקרה שבו המשחק נגמר בתיקו יוחזר $Mark.BLANK$.

3.7.4. מיד לאחר כל תור, יש לקרוא למתודה $renderBoard$ של ה-renderer להצגת הלוח.
גם אם צריך להכריז על מנצח, אנחנו מצפים שקודם יוצג הלוח, ולאחר מכן תכריזו על המנצח.

לדוגמה: אם גודל הלוח הוא 4, כלומר 4x4, וה-$winStreak$ הוא 3, אז לוח מנצח יכול להיראות כך:

X	X	X	O
		O	

(שחקן X ניצח כיוון שיש לו רצף של 3 בשורה הראשונה)

3.8. המחלקה Tournament:

תפקיד המחלקה Tournament הוא להריץ טורניר של מספר משחקים.

המחלקה מבצעת סדרה של משחקי איקס עיגול (סיבובים) בין שחקנים מסוימים בממשק רינדור מסוים, כאשר:

בסיבוב הראשון, השחקן הראשון משחק כ-X והשני כ-O, ובסוף כל סיבוב הם מחליפים סימנים. באופן הזה, בסבבים עם אינדקס זוגי השחקן הראשון X, ואילו באינקדסים האי-זוגיים זה השני.

כדי לבצע את תפקידה, המחלקה זקוקה רק לשיטה אחת, מלבד הבנאי. המחלה גם תכיל את המתודה main של התוכנית.

ה-API של המחלקה:

משמעות	מתודה
בנאי	**Tournament(int rounds, Renderer renderer, Player player1, Player player2)**
נקראת ע"י main. במתודה זו תתרחש כל הלוגיקה של הסיבובים וקריאות למשחק.	**void playTournament(int size, int winStreak, String playerName1, String playerName2)**
The main method	**Public static void main(String[] args)**

<u>הנחיות:</u>

3.8.1. שמות השחקנים בטורניר הם הסוגים שלהם, והם נמצאים בארגומנטים של שורת ההרצה.

3.8.2. בסיום כל טורניר, מודפסת על המסך התוצאה העדכנית, באופן הבא:

```
######### Results #########
Player 1, [player_type] won: _ rounds
Player 2, [player_type] won: _ rounds
Ties: _
```

את השורה האחרונה יש להדפיס בתור שורה (println), ולא לבצע ירידת שורה נוספת אחריה.

לדוגמא:

```
######### Results #########
Player 1, smart won: 687 rounds
Player 2, whatever won: 271 rounds
Ties: 42
```

3.8.3. הקפידו להדפיס את ההודעות במדויק, שינוי בתו בודד, ירידות שורה, או רווחים עלול להפיל את הטסטים.

3.8.4. יש להדפיס הודעת ניצחון רק בסוף הטורניר כולו ולא להדפיס הודעות בסיום כל משחק.

3.9. מפעלים

המפעלים יהיו אחראים על יצירת השחקנים וממשקי הרינדור. בעזרתם אנו אוכפים את "עקרון האחריות הבודדת". זאת דרך פשוטה ואלגנטית להשאיר לעצמינו את האפשרות להוסיף בעתיד עוד שחקנים ואפשרויות רינדור נוספות, כשכל מה שצריך לשנות הוא המפעל המתאים ולא שום חלק אחר בקוד.

הנחיה: המפעלים יכולים לקבל קלט לא תקין (כלומר מחרוזת שלא מייצגת טיפוס קיים), במקרה כזה הם יחזירו null כפי שלמדנו בשיעור.

בהמשך הקורס נלמד על Exceptions, שהם דרך ההתמודדות הטבעית עם שגיאות בקלט למפעל. בתרגיל זה לא למדנו את החומר עדיין, ולכן החזירו null.

3.9.1. `PlayerFactory`

המפעל `PlayerFactory` אחראי למפות את המחרוזת משורת הפקודה לאובייקט שחקן ממשי.

3.9.1.1. ה-API של המחלקה:

משמעות	מתודה
בנאי.	**`PlayerFactory()`**
תיצור ותחזיר טיפוס מסוג השחק המתאים לפי המחרוזת type.	**`public Player buildPlayer(String type)`**

3.9.1.2. המחרוזות האפשריות לבנייה הן אותן המחרוזות שמתקבלות כפרמטרים לתוכנית, כלומר human, whatever, naive, smart.

3.9.2. `RendererFactory`

המפעל `RendererFactory` אחראי למפות את המחרוזת משורת הפקודה לממשק הרינדור המתאים.

3.9.2.1. ה-API של המחלקה:

משמעות	מתודה
בנאי.	**`RendererFactory()`**
תיצור ותחזיר טיפוס מסוג הרינדור המתאים לפי המחרוזת type.	**`public Renderer buildRenderer(String type, int size)`**

3.9.2.2. המחרוזות האפשריות לבנייה הן אותן המחרוזות שמתקבלות כפרמטרים לתוכנית, כלומר console, void.

4. הנחיות והערות:

4.1. הטסטים יבחנו את התרגיל שתגישו בטורניר של כ-10,000 משחקים מעל לוח בגודל 4, ועם רצף ניצחון של 3:

- מצופה שלאחר 10,000 משחקים `SmartPlayer` ינצח את `WhateverPlayer` ברוב מוחלט של המשחקים (80% מהמשחקים).
- מצופה שלאחר 10,000 משחקים `SmartPlayer` ינצח את `NaivePlayer` ברוב מוחלט של המשחקים (80% מהמשחקים).

4.2. זכרו לעבור על מסמך coding-style הנמצא באתר הקורס. ההנחיות במסמך מחייבות.

4.3. בחרו שמות אינפורמטיביים עבור המשתנים, מתודות, קבועים והודעות ההדפסה.

4.4. אין לשנות את ה-API (פונקציות public/protected/default שאינן בתרגיל) של הממשקים ושל המחלקות שהוגדרו בהוראות התרגיל.

4.5. ניתן להגדיר פונקציות עזר ושדות פרטיים או קבועים פרטיים לפי הצורך.

4.6. על שינוי ה-API או על הוספת מתודות/שדות/קבועים שאינם פרטיים יחול קנס משמעותי. עליכם לפתור את התרגיל בלי להוסיף אותם.

4.7. **למרות זאת, <u>ניתן</u>** להוסיף **<u>מחלקות</u>** משל עצמכם, וניתן לכתוב בהן פונקציות, שדות וקבועים פומביים. עם זאת, יש להוסיף ב-README הסבר מה המחלקות שהוספתם ומדוע. הסבר זה ייבחן, וייתכן שתאבדו נקודות במקרה אינו מוצדק.

4.8. אסור להשתמש במגדיר נראות כמו עליהם לא למדנו עדיין (default או protected). כלומר במהלך התרגיל מותר להשתמש רק ב private ו-public.

4.9. תזכורת: כל הקוד הפומבי בתרגיל צריך להיות מתועד היטב (מחלקות, מתודות ושדות).

טיפ: כדאי להתחיל במימוש המחלקות הנצרכות להרצת משחק יחיד, לאחר מכן אנו ממליצים להריץ משחק יחיד בין שני שחקנים אנושיים וכך לבדוק את עצמכם תוך כדי עבודה - מתכנת טוב הוא מי שתופס את הבאגים מוקדם ולא מי שמתכנת בלי באגים.

5. הוראות הגשה

5.1. קובץ ההגשה והקבצים

עליכם להגיש קובץ zip/tar/jar (לבחירתכם) בשם ex1 (סיומת בהתאם) המכיל את הקבצים הבאים:

Game.java	.5.1.1
Mark.java	.5.1.2
Board.java	.5.1.3
Tournament.java	.5.1.4
Player.java	.5.1.5
Renderer.java	.5.1.6
PlayerFactory.java	.5.1.7
RendererFactory.java	.5.1.8
HumanPlayer.java	.5.1.9
WhateverPlayer.java	.5.1.10
NaivePlayer.java	.5.1.11
SmartPlayer.java	.5.1.12

5.1.13. `VoidRenderer.java`

5.1.14. קובץ ה-`README`

5.2. קובץ ה-README

5.2.1. בשורה הראשונה בקובץ יופיע שם המשתמש CSE שלכם.

5.2.2. בשורה השניה מספר תעודת הזהות.

5.2.3. השורה השלישית ריקה.

5.2.4. בנוסף, ענו בקובץ על השאלות הבאות לפי הסדר:

5.2.4.1. פרטו מהי האסטרטגיה שמימשתם עבור כל אחד מהשחקנים האוטומטיים.

5.2.4.2. הסבירו - מה היתרון בעיצוב התוכנה באופן שבו כל אחת ממחלקות השחקנים מממשת ממשק משותף? ציינו בתשובתכם על איזה עמודי תווך של OOP מתבסס העיצוב.

5.2.4.3. אם הוספתם מחלקות נוספות - ציינו מה תפקידן ומדוע הוספתם אותן.

6. בדיקת ההגשה

6.1. לאחר שהגשתם את התרגיל בתיבת ההגשה הקוד שלכם יעבור טסט presubmit ויווצר הקובץ results.txt.

6.2. וודאו שהקובץ נוצר בהצלחה.

6.3. הקובץ מכיל פלט של הטסט המוודא שהקוד שלכם מתקמפל, ומפרט על שגיאות בסיסיות. השתמשו בפלט שבקובץ על מנת לתקן שגיאות בתרגיל שימנעו מאיתנו להריץ את הטסטים הסופיים (זהו טסט קדם הגשה ולא הטסט הסופי של התרגיל).

6.4. ניתן להגיש את התרגיל שוב ושוב ללא הגבלה עד למועד ההגשה (ההגשה האחרונה היא ההגשה הסופית).

6.5. **שימו לב:** על פי נהלי הקורס חובה לעבור את הטסט ה-presubmit ללא שגיאות. קובץ הגשה שלא עובר בהצלחה את הטסט יקבל ציון 0 ולא ייבדק!

6.6. ניתן לחלופין להריץ ישירות את ה-presubmit על ידי הרצת הפקודה הבאה (במחשבי בית הספר):

```
~oop1/ex1_presubmit <path to your file>
```

שימו לב שפקודה זו **לא מגישה** את התרגיל בפועל אלא רק מריצה את ה presubmit.
חובה לעבור תמיד גם על הפלט של results.txt לאחר ההגשה בתיבת ההגשה לוודא שהכל תקין!

בהצלחה!